Advanced Retrieval & RAG

RAG as a Recommendation System

Bryan Bischof

4 Mar 2026

Today's Journey

Last week: **the parts** — BM25, dense retrieval, hybrid search, HyDE.

This week: **the system.**

The Through-Line

RAG is a recommendation system. Every advanced retrieval technique is an improvement to the Collector, the Ranker, or the Server.

Today's sections:

1. Recap: Unit 6
2. How this connects
3. What is a recommendation system?
4. A brief history (6 eras)
5. The Collector
6. The Ranker
7. The Server
8. Putting it all together

Outline

1. Recap: Unit 6
2. How This Connects
3. What Is a Recommendation System?
4. A Brief History
5. The Collector: Generating Representations
6. The Ranker: Matching Intent to Documents
7. The Server: From Ranked Results to Grounded Answers
8. Putting It All Together

Why Retrieval?

LLMs have two knowledge gaps:

Temporal gap

Training cutoff — nothing after that date exists for the model.

Access gap

Private, proprietary, or domain-specific data was never in training.

RAG bridges both **at inference time**: retrieve relevant documents, put them in the prompt, generate a grounded answer. No retraining required.

The alternatives we compared:

- **Retraining:** too expensive, doesn't update easily
- **Fine-tuning:** teaches style, not facts
- **Stuffing context:** doesn't scale

The easiest way to ensure a model will hallucinate is to ask it questions about resources it doesn't have access to.

The Search Abstractions

A search system has three core abstractions:

1. **Documents:** objects with named fields stored in collections. Represented as dataclasses with typed fields.
2. **Queries:** expressions of user intent — unstructured (a string) or structured (field-specific matches, range filters).
3. **Ranking functions:** the heart of search — how you score and order results.

Two search paradigms:

Lexical (BM25)

Preprocess, tokenize, build inverted index, score with BM25 (term frequency saturation + length normalization). 30 years old; no information compression; exact keyword match.

Semantic

Encode as dense vectors, find nearest neighbors via ANN (HNSW). Handles synonyms and paraphrases; can miss exact-match requirements.

Hybrid Search: Four Merge Strategies

Hybrid search runs both BM25 and dense search in parallel and merges:

Strategy	Description
Set union	Combine document IDs. Simplest; fine when order doesn't matter to an agent — you just want to cover relevant docs.
Rank-based (RRF)	Merge ranked lists using rank position only. $\mathrm{RRF}(d) = \sum_\ell \frac{1}{K+r_\ell(d)}$. Avoids score-normalization problem entirely.
Threshold-based	Weighted linear combination of normalized scores (e.g. $0.7\times$vector $+$ $0.3\times$FTS). Simple and tunable — but weights are corpus-specific.
Model-based reranking	Second-stage model rescores candidates. Cross-encoders (query + document jointly). Expensive; applied to small candidate sets.

Threshold-based vs. rank-based is not just a speed tradeoff: rank-based discards score confidence; threshold-based preserves it. A document ranked #1 with score 0.98 is probably better than one ranked #1 with score 0.51 — but RRF treats them identically. You need evals to know which wins on your corpus.

Beyond the Query

The most powerful retrieval techniques operate *around* the query:

- **Agentic search:** the LLM rewrites and iterates on queries rather than passing the user's raw text to the search engine. Decomposes intent; issues targeted, domain-specific queries.
- **HyDE:** generate "what questions could this document answer?" and embed those. Aligns document representations with likely user queries.
- **Non-query signals:** recency, authority, personalization, search history — ranking signals that exist independent of query text.

P^3 lens

The **retrieval bottleneck**: RAG quality is limited by retrieval quality. If you retrieve the wrong documents, the LLM will confidently synthesize the wrong answer. In P^3 terms: the Promise is grounded answers; the Proof requires measuring retrieval quality (precision@k, recall@k); Production requires baselines (BM25-only, random documents) to quantify how much value the retrieval pipeline adds.

Outline

1. Recap: Unit 6
2. How This Connects
3. What Is a Recommendation System?
4. A Brief History
5. The Collector: Generating Representations
6. The Ranker: Matching Intent to Documents
7. The Server: From Ranked Results to Grounded Answers
8. Putting It All Together

Unit 6 Was About Parts — Unit 7 Is About the System

Unit 6: here's how BM25 works, here's how embeddings work, here's how interleaving merges results. Each technique presented on its own terms.

Unit 7: now that you have the parts, the questions change:

- How do you **decide which parts to use** for a given problem?
- How do you **compose them** into a production architecture?
- How do you **measure** whether the composition is working — not just whether individual components are working?
- What happens when the obvious approach (**one embedding, cosine similarity**) fails — and why does it fail?
- What does retrieval look like at **real scale** — not 100 documents, but millions?

The organizing framework

We'll borrow from **recommendation systems** — a field solving exactly this "compose parts into a system" problem for decades. Netflix, Spotify, Amazon all use this framework. Today you'll learn to build with it.

This Week's Key Idea

RAG is a recommendation system.

That claim needs unpacking — especially if you haven't built a recommender before. So let's start with a primer.

Once you see this mapping, the landscape of "advanced RAG techniques" will feel *almost* obvious. Every technique we cover today is an improvement to the Collector, the Ranker, or the Server.

Outline

1. Recap: Unit 6
2. How This Connects
3. What Is a Recommendation System?
4. A Brief History
5. The Collector: Generating Representations
6. The Ranker: Matching Intent to Documents
7. The Server: From Ranked Results to Grounded Answers
8. Putting It All Together

The Core Problem

A recommendation system solves a deceptively simple problem:

Given a collection of things that may be recommended, choose an ordered few for the current context and user that best match according to some objective.

Netflix recommending movies. Spotify surfacing playlists. Amazon ranking products. TikTok assembling a For You page. At the heart of 4 out of 5 FAANG companies lies one or many recommendation systems.

The core data structure is the **user-item matrix** (*BPRS* Ch. 2) — a huge, sparse table where rows are users, columns are items, entries are preference signals (explicit ratings, implicit clicks, or nothing).

Fundamental question: predict how much a user will like an item they've never seen.

The Cheese-Tasting Matrix

Five friends rate four cheeses on a 1–5 scale. Not every cheese gets tried:

	Gouda	Chèvre	Emmentaler	Brie
A	5	4	4	1
B	2	3	3	4.5
C	3	2	3	4
D	4	4	5	**?**
E	3	**?**	**?**	**?**

Can we predict the missing entries? Would D like Brie? What should E try next?

Two natural observations:

- Which *rows* (users) are similar? A and D both rate Gouda and Chèvre highly — they might have similar taste.
- Which *columns* (items) are similar? Chèvre and Emmentaler get nearly identical ratings from everyone — they might appeal to the same people.

User-User vs. Item-Item Collaborative Filtering

User-user CF: similar rows

Users with similar taste will continue to have similar taste.

D looks like A, and A hated Brie — so maybe D won't like Brie either.

In RAG: **similar queries need similar documents.** Agentic search (Unit 6) exploits this — the agent rewrites the query to be more like queries that have worked before.

Item-item CF: similar columns

Items with similar fans will continue to attract similar fans.

E liked Gouda. Emmentaler gets similar ratings to Gouda — recommend Emmentaler to E.

In RAG: **similar documents answer similar queries.** HyDE exploits this — rewriting documents into the language users search with.

These two strategies — model the user (rows), or model the item (columns) — are the **dual engines of every recommender** (*BPRS* Ch. 2–3). They reappear throughout this lecture mapped onto RAG.

The Collector → Ranker → Server Architecture

Every recommendation system decomposes into three components (*BPRS* Ch. 0,1):

Collector

Knows the catalog; manages availability and eligibility; precomputes features. Operates mostly *offline* (batch) with an *online* component (real-time updates).

The waiter who checks what's still on the menu.

Ranker

Orders items by relevance to the current user and context. This is where the model lives. Operates *online* at request time.

The waiter ranking desserts for your tastes.

Server

Applies business logic (filtering, deduplication, diversity, permissions), satisfies the response schema, returns the final set.

Presenting 3 coherent options, not 50 raw items.

Even the simplest recommender (return a random available item) fits this framework. And the most sophisticated (Netflix's personalized homepage) uses the same three components, scaled up.

Mapping C→R→S to RAG

Component	Recommendation system	RAG system
Collector	Know the item catalog; precompute features; manage availability	Know the document corpus; precompute embeddings and indices; manage ingestion and freshness
Ranker	Score items for the current user via a model	Score documents for the current query via retrieval + reranking
Server	Apply business logic; satisfy the API schema; return K items	Apply permissions; format context for the LLM; respect token budgets; return grounded answer

The user-item matrix becomes a **query-document relevance matrix** — equally sparse, equally the core question: *predict how relevant a document is to a query the system has never seen before.*

Every retrieval technique we cover today is an improvement to the Collector, the Ranker, or the Server. The C→R→S decomposition is not just an architecture pattern — it's an *evaluation framework* that tells you where failures come from.

Outline

1. Recap: Unit 6
2. How This Connects
3. What Is a Recommendation System?
4. A Brief History
5. The Collector: Generating Representations
6. The Ranker: Matching Intent to Documents
7. The Server: From Ranked Results to Grounded Answers
8. Putting It All Together

RAG Is 6 Years Old — But Really Decades Old

The term "RAG" was coined in 2020. The *idea* — augmenting a language model with retrieved external knowledge — is much older.

The history matters because **each milestone solved a specific problem you'll still encounter today**, and each maps to a C→R→S component.

Six eras:

1. Infrastructure (2009–2016)
2. Neural memory + retrieve-then-read (2014–2018)
3. Learned retrieval (2019–2020)
4. RAG gets a name (2020–2021)
5. RAG meets LLMs (2022)
6. Reasoning + multi-representation (2024–2025)

Three lessons from the history:

1. The ideas predate the hype
2. The durable pattern: retrieve → augment → generate
3. Build a working model quickly and iterate

Era 1: The Infrastructure (2009–2016)

BM25 (Robertson & Zaragoza, 2009) *Ranker*

The probabilistic relevance framework still underpinning Elasticsearch, Solr, and most production search. Solved two TF-IDF problems:

- Term-frequency saturation: the 100th mention of a word shouldn't count 100× more than the 1st
- Document-length normalization

30 years later: still a strong baseline because it does *no information compression* — exact keyword matching.

HNSW (Malkov & Yashunin, 2016) *Collector*

Hierarchical navigable small-world graphs for approximate nearest-neighbor search. Made it practical to search millions of vectors in milliseconds.

Now the dominant index algorithm in FAISS, Pinecone, Weaviate, etc.

Without HNSW, dense retrieval at scale wouldn't be feasible.

Era 2: Neural Memory and Retrieve-Then-Read (2014–2018)

Memory Networks (Weston et al., 2014)
Collector concept

Breakthrough idea: **separate knowledge storage from computation**. A neural network with a discrete external memory it can read from and write to. The intellectual ancestor of every RAG system — the realization that a model's parameters shouldn't have to store all knowledge.

DrQA (Chen et al., 2017)
Collector + Ranker

First retrieve-and-read pipeline: TF-IDF retriever over Wikipedia + neural reader for answer extraction. Treated Wikipedia as a giant non-parametric memory for QA. The template every modern RAG system follows.

Wizard of Wikipedia (Dinan et al., 2018)
Server

Extended to dialogue: agent retrieves a paragraph and conditions on it to generate a knowledgeable response. The Server component appears: *context construction matters, not just retrieval.*

Era 3: Learned Retrieval (2019–2020)

Paper	Year	Breakthrough	C→R→S
ORQA (Lee et al.)	2019	First end-to-end *learned* retriever; treats retrieval as a latent variable problem; trains from QA pairs alone without labeled evidence passages	Ranker
DPR (Karpukhin et al.)	2020	Two BERT encoders trained on QA pairs; outperformed BM25 by 9–19% on passage recall; became the standard dense retriever	Collector + Ranker
ColBERT (Khattab & Zaharia)	2020	Late interaction: keep all token-level representations; MaxSim operator. Balanced cross-encoder richness with bi-encoder efficiency	Collector + Ranker
REALM (Guu et al.)	2020	Integrated retrieval into pre-training itself; backpropagates training loss through the retrieval step across millions of documents	Joint

Note: chunking documents into smaller pieces is the *other* architectural response to the single-vector problem that ColBERT's late interaction solves internally. Both strategies address the same information-loss problem at different levels — external decomposition vs. internal representation.

Era 4–5: RAG Gets a Name, Then Meets LLMs (2020–2022)

Era 4 (2020–2021):

- **RAG** (Lewis et al., 2020): coined the term. BART generator + DPR Wikipedia index. RAG-Sequence and RAG-Token architectures. Formalized retrieve → augment → generate. *Full system.*
- **FiD** (Izacard & Grave, 2021): Fusion-in-Decoder. Encode each passage independently; concatenate for decoder cross-attention. Use 50+ passages simultaneously — a huge advance for complex questions requiring evidence from multiple sources. *Server (context fusion).*

Era 5 (2022):

- **Atlas** (Meta): small model (11B) + retriever beats large parametric models (540B+). "Smaller but grounded." Just re-index; no retraining. *Full system.*
- **WebGPT** (OpenAI): first browser-augmented LLM. Proved live information dramatically improves factual accuracy. Covered in Unit 5. *Ranker (agentic).*
- **ReAct** (Yao et al.): Thought → Action → Observation → repeat. Template for agentic RAG. Covered in Unit 5. *Ranker (agentic).*

Era 6: Reasoning and Multi-Representation (2024–2025)

Promptriever / Rank1
(Weller et al., 2024)
Ranker (reasoning)

Made retrieval models instruction-following (Promptriever) and reasoning-capable (Rank1). Retrieval models that understand natural-language instructions and generate auditable reasoning chains. Covered in depth in the Ranker section.

GraphRAG
(Microsoft, 2024)
Collector (structured)

LLM extracts entities and relationships into a knowledge graph for structured retrieval. Influential but often over-prescribed. Covered in the Server section.

The durable pattern

The Collector got better (BM25 → dense → late interaction → multiple representations). The Ranker got smarter (fixed → learned → agentic → reasoning). The Server got more sophisticated (simple concatenation → fusion-in-decoder → context engineering).

Timeline at a Glance

Year	Milestone	Breakthrough idea	C→R→S
2009	BM25	Probabilistic term matching with saturation	Ranker
2014	Memory Networks	Separate knowledge storage from computation	Collector
2016	HNSW	Fast ANN search at scale	Collector
2017	DrQA	Retrieve-and-read over Wikipedia	Collector + Ranker
2018	Wizard of Wikipedia	Retrieval-grounded dialogue	Server
2019	ORQA	End-to-end learned retriever	Ranker
2020	DPR	Simple, effective dense retrieval	Collector + Ranker
2020	ColBERT	Late interaction (no pooling)	Collector + Ranker
2020	REALM	Retrieval integrated into pre-training	Joint
2020	RAG	Coined the term; end-to-end framework	Full system
2021	FiD	Fusion-in-decoder for many passages	Server
2022	Atlas	Small model + retriever beats large model	Full system
2022	WebGPT / ReAct	Browser-augmented and agentic retrieval	Ranker
2024	Promptriever / Rank1	Instruction-following and reasoning retrieval	Ranker
2024	GraphRAG	Entity/relationship extraction for structured retrieval	Collector

Lessons from the History

1. **The ideas predate the hype.** Memory Networks (2014) and DrQA (2017) were doing RAG before the acronym existed. Understanding the lineage helps you evaluate whether a "new" technique is genuinely novel or a rediscovery.
2. **The durable pattern is retrieve → augment → generate.** Every milestone is a variation on this loop. The specifics change (what you retrieve, how you augment, what generates) but the pattern remains consistent. The C→R→S framework gives you a principled way to decompose it further.
3. **Build a working model quickly and iterate** — the lesson from the Netflix Prize (*BPRS*). The winning solution took so long that business circumstances changed and it was no longer useful. Ship hybrid BM25 + dense, measure, improve.

Everything below is an improvement to one of the three components. When you encounter a new "RAG technique," your first question should be: which component does it improve, and how?

Outline

1. Recap: Unit 6
2. How This Connects
3. What Is a Recommendation System?
4. A Brief History
5. The Collector: Generating Representations
6. The Ranker: Matching Intent to Documents
7. The Server: From Ranked Results to Grounded Answers
8. Putting It All Together

Feature Engineering Is Not Dead

There's a tempting narrative in the LLM era: embeddings handle everything, feature engineering is a relic. **This is wrong.**

Representation engineering is the discipline of deliberately shaping how your items and queries are encoded so that semantically relevant things end up close together in the spaces you actually search.

Why this matters:

1. **Models expect different input than your raw data.** An embedding model trained on web search data expects natural prose, not your internal Jira ticket format or 200-page regulatory filing. Reshape your data to match what the model was trained on.
2. **Representation shapes what knowledge can be inferred.** A financial report embedded as one blob loses the distinction between revenue data, risk factors, and management commentary.
3. **Scale problems hit in two dimensions.** Document stores grow (10K $\rightarrow$ 10M records) *and* documents are often larger than context windows. You need to find the right 3 paragraphs from the right 5 documents out of 10 million. A single similarity search can't do that reliably.
4. **Precision matters because context poisoning exists.** Good features = shorter, more precise context = fewer wrong paths.
5. **Personalization requires features.** "Good answer" is subjective without user context.

The Central Insight of the Collector

You can only find what you've made findable.

A movie described only by genre is matchable only by genre. A movie described by genre + mood + era + director style is matchable on all of those dimensions. A document embedded as a single vector is matchable only on whatever the model decided to prioritize.

Every representation decision is a decision about what questions can be answered.

The map is not the territory. You can create many different maps of the same territory, each optimized for different queries. A single embedding is one map. Multiple representations are multiple maps.

Recsys analogy

Netflix represents each movie with collaborative features (who watched it), content features (genre, director, mood), visual features (thumbnail similarity), and text features (plot description). Different users match on different dimensions.
In RAG: different users express the same information need in different ways. Build representations for all of them.

Deconstructing RAG Buzzwords

Every "advanced RAG" technique is just an answer to one of the three C→R→S responsibilities, dressed up with marketing:

Buzzword	What it actually is	C→R→S
Naive RAG	Simple single-vector search	Basic Collector + Ranker
Advanced RAG	Any RAG that improves on naive; often multi-stage or hybrid	Improvements across all three
Modular RAG	Pipeline as composable, swappable modules	Formalizes the C→R→S decomposition
Agentic RAG	LLM rewrites queries and iterates	Ranker improvement
Hybrid RAG	Multiple search signals combined	Ranker using multiple Collector outputs
Graph RAG	LLM extracts entities and relationships into a knowledge graph	Collector creates structured representation
Multi-Modal RAG	Search across text + images (or multiple index modes)	Collector with multiple representation types
Self-RAG	Model decides when to retrieve; critiques its own results via reflection tokens	Ranker (adaptive) + Server (self-critique)
Corrective RAG (CRAG)	Validates retrieved docs; falls back to web search if low quality	Server (quality gate between Ranker and generation)
Speculative RAG	Small specialist model drafts answers from different doc subsets in parallel; large model verifies	Server (draft-then-verify for latency/accuracy)
Adaptive RAG	Routes between no-retrieval, single-step retrieval, multi-step retrieval	Ranker (dynamic routing)

All of these could have been derived from first principles by asking "which C→R→S responsibility is underperforming, and how can I improve it?" The buzzwords obscure this clarity.

Multiple Representations: Many Maps of the Same Territory

Instead of one embedding per document, create several. **Semantic Dot Art** (`semantic.art`) demonstrates every Collector concept in one system.

The system indexes hundreds of thousands of artworks. Users search in wildly different ways:

- "an oil painting of the side profile of a woman surrounded by flowers" (literal description)
- "A path winds on, beneath a vibrant sky. Sun-warmed grasses whisper secrets low." (poetic mood)
- "nature, solitude, dream, deep" (mood keywords)
- or just an image

A single embedding can't serve all of these. The Collector creates **multiple representations** of each artwork:

Representation	What it captures	Example (Van Gogh's Path Through a Field with Willows)
Poetic caption	Mood and emotion as prose	"A lonely journey, bathed in golden light. Nature breathes, both calm and wild."
NL caption	Literal + evocative description	"A path meanders under a bright sky, where sun-warmed grasses softly rustle..."
Mood keywords	Emotional and thematic tags	`nature, solitude, dream, deep, intensity, path`
Image embedding	Visual similarity	The image itself, embedded via a vision model

Each representation gets its own index. The same artwork appears in all of them, but each "map" captures a different facet of what makes it matchable.

Multiple Representations: Domain Patterns + PinSage

The general pattern applies to any domain:

Domain	Representations you might create
Artworks	Poetic caption, NL caption, mood keywords, image embedding
Financial docs	Summary, data tables, entity lists, form types
Code repos	Function signatures, docstrings, README, dependency graph
Products	Title, description, attributes, review sentiment, images

The canonical production example of this thinking in recsys:

PinSage (Ying et al., 2018 — Pinterest)

Graph-based recommender combining visual, textual, and engagement signals into rich per-pin representations. Nodes are pins; edges encode co-saves (if many users saved both pin A and pin B, they're connected). One of the earliest production systems to make "multi-modal representation" standard practice.

The intuition carries directly into document representations in RAG: different users match on different dimensions. The canonical engagement signal that no single-item embedding captures.

The Semantic-Space Mismatch Issue

Semantic-space mismatch

A user opens Semantic Dot Art and types "a quiet optimism painted in the sky." That query embeds into a point in *language space*. The raw image embedding of Van Gogh's *Wheat Field with Cypresses* is in *vision space* — dense vectors derived from pixel patterns, color distributions, and visual structure. There is no reason to expect those two vectors to be close. They're not even playing the same game.

This is the fundamental mismatch problem. Your items were created in one modality, one format, one register — and your users will express intent in an entirely different one:

- Legal filing in legalese; user asks "what's our liability if a contractor gets hurt on site?"
- Financial report structured for auditors; analyst asks "is this company burning cash?"
- Medical record in clinical shorthand; patient asks "why did my doctor order that test?"

Representation engineering is how you bridge this gap. In Semantic Dot Art, we ran every artwork through a multimodal LLM to generate captions *in the language users actually use to describe what they're looking for*. The Collector builds the bridge offline so the Ranker can cross it at query time.

Document Enrichment: Creating the Maps

HyDE (Unit 6) is actually a **document enrichment** pipeline — rewriting documents into the language users search with. More broadly: any offline process that creates new representations.

In Semantic Dot Art: multimodal LLMs (Google Gemini) generate poetic and NL captions from each artwork image, and extract mood keywords. The Collector prepares the catalog so the Ranker has rich features to work with.

A critical design choice: all representations live in **the same row** of a single LanceDB table — vectors, captions, mood keywords, and JPEG bytes together.

- Adding a new representation (e.g. a color-palette embedding) = adding a column, not rebuilding the index
- The corpus keeps growing, but the storage architecture doesn't need to be refactored

The scale challenge: enriching hundreds of thousands of artworks requires distributed computation. This pattern generalizes: any document enrichment pipeline at scale is a **batch LLM data-processing job** (Unit 11). Fan-out, structured output, validation between steps.

A static embedding goes stale. If a financial report is embedded today and the company announces a major acquisition tomorrow, the embedding doesn't reflect that. The Collector's job is not just to build representations — it's to keep them fresh.

Chunking: What It Looks Like When You're an Expert

Most RAG tutorials show fixed-size chunking: split every document into 500-token blocks. **This is usually dumb.**

Chunk too aggressively:
Break macro understanding. A chunk about "Q3 revenue growth" has no idea the document's *conclusion* says growth is decelerating. Fragments individually relevant, collectively incoherent.

Embed whole document:
Lose ability to find specific sections. One vector can't represent both revenue data and risk factors.

Expert chunking:

- **Inconsistently sized chunks.** Semantically coherent sections stay together, even if one is 200 tokens and another is 2,000. Don't impose a grid on content that has natural structure.
- **Prepend the situation.** Every chunk: "From the Risk Factors section of Acme Corp's 2025 10-K filing: [chunk text]." Without this, the chunk is an orphan.
- **Chunk along document structure,** not character counts. Use headings, sections, paragraphs. A chapter break is a chunking signal; the 512th token is not.
- **Overlap when structure isn't available.** 10–20% overlap for plain text.

Chunking *is* feature engineering. Every chunking decision is a decision about what knowledge can be inferred from any atomic unit. Make those decisions with domain expertise, not with `text[:512]`.

Long-Document Strategies: Hierarchical Indexing and RAPTOR

Chunking alone breaks macro understanding. Sometimes you need *both* local detail and global context simultaneously.

Hierarchical indexing: two C→R→S pipelines in series.

- **Document-level index:** one embedding per document (or major section) — broad topic matching
- **Chunk-level index:** fine-grained pieces — precise factual retrieval

First retrieve at the document level to identify relevant documents, then at the chunk level within those documents.

RAPTOR (Sarthi et al., ICLR 2024)

Builds a *tree of summaries* at multiple granularities:

1. Chunk document into leaf nodes
2. Cluster similar chunks → summarize each cluster (level-1 nodes)
3. Cluster level-1 summaries → summarize again (level-2 nodes)
4. Repeat until a single root summary covers the whole document

At query time: search *all levels simultaneously*. A broad query matches a high-level summary; a narrow factual query matches a leaf chunk.

Key result: 20% absolute accuracy gain on QuALITY benchmark with GPT-4.

Multimodal Representations

Real enterprise corpora are rarely pure text: PDFs contain charts and figures; technical docs have architecture diagrams; financial filings mix structured data and prose.

Extract-then-embed

Convert non-text content to text (LLM or vision model), then embed. A chart becomes "Bar chart: Q1 $2.1B, Q2 $2.4B, Q3 $2.2B, Q4 $2.8B — upward trend with Q3 dip." Quality depends entirely on the extraction step.

Native multimodal: ColPali

Faysse et al. (2024). ColBERT architecture applied to document images — OCR-free. Page image → token-level visual embeddings → MaxSim. Handles charts and mixed layouts that OCR would mangle.
`arxiv.org/abs/2407.01449`

Structured/tabular data:

- **Row serialization:** "In Q1 2025, revenue was $2.1B, margin 18%, headcount 12,400." Each row is a retrieval unit.
- **NL2SQL as tool use** (Unit 5): generate and execute SQL — exact results, not approximate semantic matches.
- **Hybrid:** embed a prose summary for semantic retrieval; use SQL for precise computation once you've identified the right table.

Key principle

Match the representation to the query type. Approximate conceptual questions → embed. Exact factual

Embedding Model Selection

Not all embedding models are created equal. "Which model is best?" depends on your domain, query type, and constraints.

Starting points:

- **MTEB/BEIR leaderboards** — useful for narrowing the field but **contaminated**: models are trained on these benchmarks. Don't trust top-10 as ground truth on your data.
- **Domain match** matters more than parameter count.
- **Context length caveat:** many claim 8K tokens but degrade past 4K. Test on your documents.

Matryoshka embeddings (MRL)

Kusupati et al., NeurIPS 2022 (`arxiv.org/abs/2205.13147`). First N dimensions alone form a good representation — **truncate** without retraining. Up to **14×** **storage reduction**.

Quantization

- float32 → int8: **4×** **savings**, minimal recall drop
- float32 → binary (1 bit): **32×** **savings**, larger but often acceptable recall cost

When to fine-tune

Need ∼1K–10K positive (query, relevant passage) pairs. Evaluate on your golden dataset. **If lift is <5%, the general model is fine.**

Pre-Filtering: Narrow Before You Search

At scale, you don't want to run a vector search across 10 billion chunks. **Pre-filtering** uses structured metadata to narrow the search space *before* the embedding-based similarity search runs.

The key distinction:

Pre-filter (before search)

Reduce the index to eligible candidates, then run ANN. Faster and more precise — the similarity search focuses its budget on the right region of the embedding space.

Post-filter (after search)

Run ANN over everything, then discard non-matching results. Wasteful and can leave you with *fewer results than requested — or none at all.*

Every enriched representation is a potential new filtering dimension: mood keywords (Semantic Dot Art), entity lists, document type, author, recency, category. Each enrichment step adds structured metadata you can pre-filter with.

What metadata to expose is a Collector design decision: when you design your document schema, think about what structured dimensions users will want to narrow by. These fields should be indexed as structured columns alongside the embedding vectors.

Pre-Filtering: Code + The Empty-Context Issue

Pre-filter in LanceDB

```
results = (
    table.search(query_embedding)
        .where("department = 'finance' AND year = 2025",
               prefilter=True)
        .limit(20)
)
```

Only searches chunks where `department` is finance and `year` is 2025 — a potentially huge reduction in candidate space before any vector math runs.

The empty-context issue

A user asks "what did our legal team say about GDPR compliance last month?" Your system retrieves the top-20 most semantically similar chunks from the entire corpus, *then* filters to `department = 'legal' AND date >= '2025-02-01'`. The top-20 ANN results all come from the finance team discussing a related data privacy topic — semantically close, but wrong department. Every result is discarded.

The LLM receives an empty context and says "I don't have information about that" — or hallucinates an answer — **even though the answer exists in the index.** The document was retrievable; you just didn't narrow the search space first.

This is one of the most common silent failures in production RAG systems.

Chunking and Retrieval: A Systematic Experiment

To make these Collector ideas concrete, here's a retrieval experiment testing **9 chunking strategies** $\times$ **4 retrieval methods = 36 combinations** on a corpus of expert interview transcripts, scored against 69 curated benchmark queries.

Chunking strategies tested:

- **QA-pair chunking:** LLM extracts question-answer pairs — each pair is a chunk (document enrichment)
- **Key-question segment chunking:** LLM identifies key questions, chunks around them with surrounding context (expert chunking with situational prepending)
- **Sliding window:** overlapping fixed-size windows ("dumb but safe" baseline)
- **Fixed-token:** non-overlapping fixed-size blocks ("even dumber" baseline)
- Each strategy tested at 256- and 512-token sizes where applicable

Retrieval methods tested:

- Pure vector search (OpenAI embeddings)
- Pure full-text search (BM25)
- Threshold-based interleaving within a single index (weighted score combination)
- Multi-index interleaving (two different chunking strategies searched in parallel, merged via RRF)

Evaluation: LLM-as-judge (Gemini 2.5 Flash) alongside nDCG, MRR, Precision@k, Recall@k, HitRate@k.

What the Experiment Showed

1. **Multi-index interleaving gave the best results.** Running two different chunking strategies in parallel + RRF outperformed any single index. The key insight: you can interleave across *chunking strategies*, not just across search methods. Each strategy captures different aspects of the same documents.
2. **Threshold-based interleaving outperformed pure vector or pure FTS** within a single index — but the weights are not transferable. What worked for this conversational corpus wouldn't necessarily work for a legal corpus or a codebase. *This is where evals become essential.*
3. **Larger chunks outperformed smaller** within the same strategy family on this conversational corpus. On a corpus of short, self-contained records (support tickets, product descriptions), the opposite might hold. *Chunk size is a hyperparameter — measure it.*
4. **FTS added little over pure vector** for conversational text with few domain-specific keywords. A legal or medical corpus would almost certainly show the opposite. *Run your own eval rather than trusting someone else's findings — including these.*
5. **LLM-as-judge caught things IR metrics missed.** A chunk could score well on nDCG (right document, ranked high) but score poorly with the judge (lacked enough surrounding context to actually answer the question). Both metric types together gave a more complete picture.

Multi-Index Interleaving and Learning Weights

In practice you often want both rank-based and threshold-based interleaving — and the question becomes: **how do you find the right weights?**

Consider a system with three indices: semantic embeddings, lexical index, late-interaction entity index. You have **two layers** of interleaving decisions:

1. **Within each index:** if the index supports hybrid search (vector + FTS), what threshold weights do you use?
2. **Across indices:** how do you interleave results from all three indices into a single ranked list?

RRF is often the safest default across indices — you don't need to compare scores across fundamentally different systems. But threshold-based can outperform RRF when you have good weights: it preserves the *confidence* information that RRF discards.

Treat weights like any other hyperparameter:

1. Build your golden dataset (Units 1, 3)
2. Define your target metric (precision@k, nDCG, or composite)
3. Run grid/random search over the weight space
4. Evaluate each combination on the golden dataset
5. Pick the winner — then **re-evaluate periodically** as your corpus changes

With two indices: one weight to tune. With five — semantic, lexical, entity, HyDE, late-interaction — you have a five-dimensional weight space. The more representations you create, the more dependent you become on systematic measurement.

Online vs. Offline: The Dual-Path Pattern

Every recsys component has both an offline and an online mode:

Offline Collector

Batch-process the corpus, compute embeddings, build indices. Runs on a schedule or triggered by data changes.

Handles volume.

Online Collector

Handle real-time updates: new documents, edits, deletions. Must update indices without full recomputation.

Handles freshness.

Without both, you're either slow to onboard or stale on updates.

Notion — the Collector at consumer scale

Notion is a collaborative workspace where millions of users store notes, projects, wikis, and databases. When they launched Notion AI Q&A, the product needed to answer natural-language questions like "what did we decide in last week's product review?" by searching across a user's entire workspace. Keyword search fails here because users don't know the exact words in the target document. Vector search lets Notion match intent to content semantically across heterogeneous, constantly-changing workspaces.

Notion Case Study: Why RAG + Dual-Path + Time

Engineering challenge: each workspace is isolated (users can only search their own content), workspaces range from one student's notes to enterprise-scale, and pages are edited continuously in real time.

The dual-path Collector:

- **Offline path:** batch jobs chunk existing documents, generate embeddings, bulk-load vectors. Used for initial workspace onboarding and model migration.
- **Online path:** a streaming consumer processes individual page edits in real time at sub-minute latency — new pages, deleted blocks, permission changes.

Time as a first-class dimension: Notion tracks when each chunk was last modified. This makes recency a pre-filter and a non-query ranking signal — pages edited this week rank higher for a query about "recent decisions" than a page from two years ago, even if both are semantically similar.

Without time in your metadata, you can't distinguish "what does our policy say?" (any version is fine) from "what is the *current* policy?" (recency matters critically). Time is almost always one of the most valuable metadata fields to expose.

Notion Case Study: Smart Change Detection (70% Reduction)

The trickiest Collector problem: when a document changes, how much do you re-index? Naive answer: re-embed and reload the entire document on every edit. At Notion's scale, this was financially unsustainable.

Their solution: hash both the *text* and *metadata* of each chunk separately, and store those hashes. On every edit, diff the new hashes against the stored ones:

- **Only text changed:** re-embed and reload just the changed chunks, not the whole document
- **Only metadata changed** (e.g. permissions): skip re-embedding entirely — the vector is still valid. Issue a metadata update to the stored record.
- **Neither changed:** do nothing

Result: 70% reduction in data volume (and associated embedding API costs and index write costs).

The key insight: text content and metadata are *orthogonal concerns*. Treating them separately lets you avoid the expensive work (embedding generation) when only the cheap work (metadata update) is needed.

The index isn't a static artifact you build once. It's a living system that must handle creates, updates, and deletes efficiently. Detecting *exactly what changed* and doing only the necessary work is the engineering discipline.

The Pooling Problem: Why Single-Vector Search Fails

Recall from Unit 6: semantic search encodes documents as single vectors via an encoder + **pooling** step.

Pooling compresses all token-level information into one vector. The compression is *selective* — the model prioritizes what its training data considered important:

- Trained on movie-review data where queries ask about actors? It encodes actor names and discards plot details.
- On out-of-domain data (legal contracts, cooking recipes)? The learned notion of "important" is wrong.

Recsys analogy

Encoding movies by genre alone loses director, mood, and era. Users asking "films like Blade Runner" (genre + mood + era) get bad recommendations because the model only sees "sci-fi."

Two architectural responses:

1. **Chunk externally:** decompose documents into sub-units, each gets its own vector (solves it at the document level — this is what expert chunking does)
2. **Late interaction:** keep all token-level representations inside the model (solves it at the architecture level — this is ColBERT)

Both strategies address the same information-loss problem at different levels.

Late Interaction: Keep All the Tokens

ColBERT and late interaction models solve the pooling problem by skipping the compression step. They keep *all* token-level representations.

The **MaxSim** operator computes relevance by finding the maximum similarity between each query token and all document tokens, then summing:

MaxSim diagram

```
Query tokens:    [q_1]  [q_2]  [q_3]
                   ↓      ↓      ↓
                  max    max    max    ← max similarity against
                   ↓      ↓      ↓       all doc tokens
Document tokens: [d_1] [d_2] [d_3] [d_4] [d_5] ...

Score = max_sim(q_1) + max_sim(q_2) + max_sim(q_3)
```

Results from the course reader (*same backbone and training data*):

- Late interaction **19.61** vs. dense **12.31** on BRIGHT — architecture alone accounts for **60% improvement**
- **150M-parameter late interaction model outperforms all 7B-parameter dense models** on reasoning tasks (45× smaller)
- **Interpretability bonus:** token-level matching shows exactly which document parts contributed — useful for debugging and providing precise context to the LLM

Late Interaction: Extensions and Measuring the Collector

Additional late interaction findings:

- Late interaction models trained on documents of only 300 tokens generalize to 8K+ token contexts — the architecture handles long documents without long-document training
- **ColPali** applies the ColBERT architecture to images for OCR-free document retrieval
- Late interaction handles cold-start better than dense models because they don't compress — less opportunity for training distribution bias to discard the "wrong" information

Measuring the Collector:

What to measure	Metric	What it tells you
Can the system find relevant docs?	Recall@50, Recall@100	Upper bound on system quality — if the Collector misses a doc, the Ranker
Are the right representations being used?	Recall per index type	Which maps contribute and which are dead weight
Does enrichment help?	Recall before/after HyDE, summaries	Whether the enrichment investment pays off
How fresh are the indices?	Staleness audit (% of docs $> X$ days old)	Whether the online Collector is keeping up

P^3 lens

The Collector sets the **ceiling** for the whole system. Measure recall at a generous K (50–100) as your Collector metric — separate from the final precision@K the user sees.

Outline

1. Recap: Unit 6
2. How This Connects
3. What Is a Recommendation System?
4. A Brief History
5. The Collector: Generating Representations
6. The Ranker: Matching Intent to Documents
7. The Server: From Ranked Results to Grounded Answers
8. Putting It All Together

The Ranker's Job

The Ranker takes what the Collector prepared and scores it for the current query.

In recsys terms: user modeling → candidate generation → scoring → reranking. *In RAG:* query understanding → retrieval → reranking.

Query understanding is the Ranker's first stage: transforming a raw query into something retrieval can act on. In a recommendation system, user modeling is everything — you need to know what someone wants before you can find it. In retrieval, the equivalent is query understanding.

Unit 6 introduced two approaches:

- **Agentic search:** the LLM rewrites queries in a loop
- **Structured queries:** field-specific, typed queries

Both are *user-side collaborative filtering* — the system infers what the user really needs, which may differ from what they typed.

CF mapping

User-user (query-side): similar queries need similar documents. Agentic search exploits this — the agent rewrites the query to be more like queries that have worked before.
Item-item (document-side): similar documents answer similar queries. HyDE exploits this — rewriting documents into the language users search with.

Semantic Dot Art: The Full Query-Understanding Pipeline

When a user searches for art, the Ranker:

1. **Classifies the query** as "natural" or "poetic" via LLM agent → determines which representation index to search (query understanding → routing)
2. **Generates an image caption** if the user provided an image, in the appropriate style
3. **Rewrites the query** combining text and image caption into the target style (query rewriting)
4. **Extracts mood keywords** from the rewritten query for prefiltering (structured enrichment)
5. **Prefilters** the index to only artworks sharing at least one mood keyword (fast candidate reduction)
6. **Searches** the appropriate representation index — full-text, vector, or hybrid depending on query length
7. **Reranks** results using a weighted combination of mood-keyword precision and recall between the query and each artwork

This is all three classic search techniques — query understanding, query rewriting, multi-index search — orchestrated by an LLM agent. The agent is the **translation layer** between raw human expression ("Where may I find a piece full of grief and woe?") and the structured representations the Collector prepared.

Semantic Dot Art: The Search Call

After all that routing and rewriting, the actual search call is compact:

Hybrid search with pre-filter and custom reranker

```
results = (
    table.search(
        query_type="hybrid",
        vector_column_name="poetic_vector",
    )
    .vector(query_embedding)
    .text(query_text)
    .where(prefilter_clause, prefilter=True)
    .limit(10)
    .rerank(CustomKeywordRanker(keywords))
)
```

Hybrid search, prefiltering on mood keywords, and custom reranking in one chained call. The **complexity lives in the routing** (which column to search, which query to use), not in the retrieval API itself.

The Collector builds the maps. The Ranker decides which map to consult for each query. The LLM agent is the router between them.

Promptriever: Instruction-Following Retrieval

A limitation of query rewriting: it happens *outside* the retriever. The retriever itself is a dumb function — takes a string, returns nearest neighbors. It can't understand instructions like "find documents that use metaphors" or "prioritize recent sources."

Promptriever (Weller et al.) changes this. A bi-encoder trained on *instruction negatives* — documents relevant to the query but irrelevant to the instruction. The model encodes instructions directly into the query embedding.

Results:

- First bi-encoder with positive scores on instruction-following benchmarks
- **Zero-shot hyperparameter optimization via prompting:** tell the model "have really high recall" in natural language and it adjusts its internal strategy

Recsys analogy

Promptriever is a recommendation system where the user can say "show me adventurous picks, not safe ones" and the *ranking function itself* changes behavior. Previously you had to retrain to change retrieval behavior; now you prompt the retriever directly.

Rank1: Reasoning at Reranking Time

Rank1 (Weller et al.) applies test-time compute to reranking. A cross-encoder that generates explicit reasoning chains to assess relevance — identifying key phrases, analyzing relationships, questioning its own interpretations.

Results:

- Nearly **doubles** baseline on reasoning benchmarks (BRIGHT) and negation tasks (NevIR)
- **10-point gain** just from training with thinking traces vs. without — learned via distillation from DeepSeek-R1, not RL
- Discovers **novel relevant documents** that previous systems missed entirely: when researchers re-judged previously unjudged docs from DL19/DL20, Rank1 became the top-performing model

Recsys analogy

Rank1 is a recommendation system that *explains its ranking decisions*. The reasoning chain is auditable — you can understand why a document was ranked first. This is the recsys equivalent of "show your work."

Multi-Stage Ranking (Same as Recsys)

Production recommendation systems almost always use multi-stage ranking (*BPRS* Ch. 12): a fast, cheap candidate generator followed by a slow, expensive ranker. Retrieval works the same way:

```
Full corpus (millions)
  → Stage 1: fast retrieval (BM25 + dense, top 100-500)
    → Stage 2: cross-encoder reranker (top 10-50)
      → Stage 3 (optional): reasoning reranker (Rank1, top 5-10)
```

Each stage trades throughput for accuracy. The merge strategies from Unit 6 (interleaving/RRF, set union) handle Stage 1. Stages 2–3 can now include reasoning — impossible before Promptriever and Rank1.

Instacart: LLM-Powered Query Understanding

Instacart's search serves millions of product queries, many long-tail and ambiguous: "bread no gluten", "x large zip lock", "healthy snacks for kids lunch".

Their old system used *separate traditional ML models* for each query-understanding task (spell-check, synonym expansion, intent classification, query rewriting) — creating system complexity and poor performance on rare queries due to data sparsity.

Their new **Intent Engine** architecture:

- **Query understanding (Ranker stage 0):** a single LLM call replaces multiple specialized models, handling spell correction, synonym expansion, intent classification, and query rewriting in one step. The LLM generalizes to long-tail queries that individual models couldn't handle.
- **Multi-stage retrieval (Ranker stages 1–2):** the rewritten query feeds into lexical + semantic search, then reranking. The LLM doesn't replace retrieval — it *improves what the retrieval system sees*.
- **Multimodal enrichment (Collector):** their PARSE system extracts product attributes from text *and* images, so the Collector has rich features for the Ranker to match against.

The biggest retrieval gains often come from better *query understanding*, not better embeddings.

BM25 Still Works — And Now We Know Why

BM25 — a 30-year-old algorithm — outperforms many dense models on long-context and reasoning benchmarks. The explanation maps directly to the pooling problem:

BM25 does *no compression*. It matches exact keywords without losing information. There's no model that can prioritize the "wrong" dimensions, because there's no model.

BM25 fails when there's no lexical overlap (synonyms, different languages). Hybrid search — combining BM25 with semantic methods — consistently outperforms either alone. *Complementary failure modes.*

Recsys analogy

BM25 is "filter by exact attribute" — ignores subtlety but never misses a direct match. The user who says "pepperoni pizza" gets pepperoni pizza.

Cold-Start: The Retrieval Version

In recsys, cold-start is when a new user arrives with no history, or a new item has no interactions. Encoder architectures and feature-based warm starting address this (*BPRS* Ch. 7).

In retrieval, cold-start appears as:

- **New document types** the embedding model has never seen (a codebase when trained on web text)
- **New query patterns** from a new user population
- **Domain shift** when the system is applied to a new industry

Late interaction handles cold-start better

They don't compress — there's less opportunity for the training distribution bias to discard the "wrong" information. Token-level matching is more robust to domain shift.

BM25 handles cold-start perfectly

It requires no training at all. When you deploy a system to a new domain, BM25 is your most reliable baseline — and often your best performer until you can collect enough domain-specific data.

Ranking for the LLM, Not for a User

In classical search, the user reads results top-to-bottom. Rank #1 gets the most attention. All standard IR metrics (MRR, NDCG, Precision@K) are built around this assumption.

For RAG, the consumer is an LLM, and LLMs don't behave like linear readers. Unit 2 introduced **lost in the middle**: models attend most strongly to the *beginning* and *end* of their context. Documents placed in the middle of a long context window are systematically underused.

This has a non-obvious consequence: **for RAG, ordinal ranking is not always what you want.**

Practical implications:

- **Be selective about K.** Passing 20 docs when 5 would do exposes the LLM to more noise and more middle-context degradation.
- **Consider position, not just rank.** The best documents should go at the *beginning or end* of the context, not in the middle — this is a Server-level concern, but the Ranker needs to identify which documents are most important so the Server can place them correctly.
- **Diversity matters more than pure relevance ordering.** If your top-3 documents all say the same thing and docs 4–6 cover different critical facets, the LLM may miss those facets entirely. A Ranker optimized for NDCG can paradoxically hurt RAG quality.

Highlighted Documents: Making Importance Explicit

One practical response to lost-in-the-middle: don't just *position* the best documents, **annotate them explicitly.**

Annotating highly relevant passages in the context

```
[HIGHLY RELEVANT - confidence: 0.94]
"The GDPR compliance policy was updated on March 1, 2025. All vendors
must complete the new DPA before April 30..."

[SUPPORTING CONTEXT]
"Our data processing team uses the following template..."
```

This is **representation engineering applied to the context window itself.** The LLM's attention is shaped not only by position but by the signal in the tokens — an explicit "highly relevant" tag is a strong prior that this passage deserves attention even if it falls in the middle.

You can derive the annotation from the Ranker's confidence score, reranker output, or a combination (e.g. top-2 results above a score threshold get the highlight; everything else is supporting context). The threshold is a hyperparameter you tune on your eval set — the same way you tune retrieval weights.

Context Rot and Context Poisoning

Kelly Hong et al. tested 18 state-of-the-art models on performance degradation as context grows (**Context Rot**, Chroma, July 2025):

- **Semantic matching degrades faster than lexical matching** as context grows
- **Distractors are devastating — this is context poisoning:** the longer the context, the higher the odds that a single misleading token sends the model down the wrong path. Semantically similar but incorrect information causes confident hallucination (GPT models worst; Claude more likely to abstain)
- **More distractors + longer context = compounding degradation**
- **Shuffled context slightly helps** — models do better when the haystack is randomly shuffled than when it's a coherent essay. LLMs don't process context linearly.
- **The U-shaped attention pattern may not hold anymore** — Chroma found no consistent advantage for placing information at start or end, qualifying the "lost in the middle" finding from Unit 2
- **No single model resists context rot across all tasks**

Three takeaways

1. Calibrate your Ranker to the context lengths you actually send.
2. Having the right information isn't enough — presentation matters.
3. **Retrieve less but better.**

Measuring the Ranker

What to measure	Metric	What it tells you
Is the best doc in top positions?	MRR	How quickly the LLM sees the first relevant result
Are the top-K results good?	Precision@K, NDCG@K	Quality of the ranked list the Server will use
Did query rewriting help?	Precision/recall before vs. after	Whether the LLM query-understanding step adds value
Does reranking help?	NDCG with/without reranking	Whether the expensive Stage 2/3 justifies its cost
Per-stage contribution	Recall@K at each stage boundary	Where in the pipeline quality is gained or lost

Critical practice: measure at each stage boundary, not just end-to-end. If Stage 1 has 60% recall@100 and final accuracy is 55%, the bottleneck is the Collector (not enough candidates), not the Ranker. If Stage 1 has 95% recall@100 but accuracy is still 55%, the Ranker is failing to sort the good candidates to the top.

P^3 lens

The Ranker is the **Proof** target. Your retrieval eval (precision@k, recall@k, coverage, diversity) measures whether the Ranker does its job. Always compare against baselines (Unit 1): BM25-only, random documents, and "stuff the whole corpus." The baseline quantifies how much value your ranking pipeline adds.

Outline

1. Recap: Unit 6
2. How This Connects
3. What Is a Recommendation System?
4. A Brief History
5. The Collector: Generating Representations
6. The Ranker: Matching Intent to Documents
7. The Server: From Ranked Results to Grounded Answers
8. Putting It All Together

The Server's Job

In a recommendation system, the server isn't just "return the top K." It applies business logic (inventory constraints, hard avoids, diversity rules), satisfies the response schema, and ensures the final set of recommendations is coherent.

In RAG, the server component is the bridge between "here are the top-K retrieved documents" and "here's a grounded answer for the user." It is often neglected — and often where the system breaks.

Orchestrator + subagents pattern

When context rot is a concern, use **orchestrator + subagents**. Each subagent gets focused context (just the relevant documents for its subtask) and returns only essential findings. This prevents context overload and keeps each subagent's context clean of distractors.

In complex multi-step tasks: a parent agent spawns subagents — one retrieves from the legal knowledge base, another from financial data, another from recent meeting notes. Each runs its own C→R→S pipeline against the relevant collection, then returns only its findings to the parent. Solves both context poisoning (each subagent gets a clean context) and permissioning (each subagent scoped to only the collections it's authorized to search).

Measuring the Server: Metrics + Diagnostic Flowchart

What to measure	Metric	What it tells you
Does the answer use retrieved evidence?	Faithfulness	Whether the LLM generates from context or hallucinates
Is the answer complete?	Coverage (% required facts in answer)	Whether the Server provided enough evidence
Are retrieved docs diverse?	Diversity (alpha-nDCG)	Whether the Server deduplicates or passes 10 copies of the same fact
Does context length hurt?	Quality at short vs. long context	Whether context rot is degrading your specific system
Does the answer cite sources?	Citation accuracy	Whether provenance is working

When end-to-end answer is wrong, trace where it broke

```
Collector missed the doc?           → Recall problem (add representations)
Ranker had it but ranked low?       → Ranking problem (improve reranker)
Doc was in context, LLM ignored it? → Context rot (reduce context, reorder)
LLM used doc but generated wrong?   → Generation problem (prompt engineering)
```

This decomposition — Collector vs. Ranker vs. Server failure — is what makes C→R→S an **evaluation framework**, not just an architecture framework. When something goes wrong, you know *where to look*.

Memory vs. Retrieval: The User Model

In a recommendation system, the **user model** is what makes recommendations personal — it tracks preferences, history, and how both evolve over time. Without it, every user gets the same results.

Stateless retrieval

Answers: "given this query, which documents are relevant?" Doesn't care who's asking. Same query → same results for every user. This is what our Collector and Ranker do.

Stateful memory

Answers: "given this user's history and current state, what additional context should influence the response?" This is the user model — preferences, past interactions, facts that have changed.

The failure mode when you use retrieval for memory:

Database migration example

Week 1: "We're using Postgres for everything, it's working great."
Week 4: "Postgres is getting slow; we're migrating to a data warehouse."
Week 6: "Migration done — Snowflake is now our source of truth."
Week 8: User asks: "What database should we query for the revenue report?"

Semantic search returns "Postgres is working great" (highest similarity to "database"). Memory system returns "Snowflake is now our source of truth" (current state). "Postgres is working great" is **temporally invalid** — superseded by the migration.

Memory vs. Retrieval: Dimensions and C→R→S

	Stateless retrieval	Stateful memory
Scoping	Same results for all users	User/entity-specific
Temporal	No concept of "when"	Tracks validity windows
Invalidation	Documents are current or deleted	Facts can be superseded without deletion
Relationships	Document-to-query similarity	Entity relationships (user→preference→brand)

A complete RAG system retrieves from memories too.

A production system typically has at least two retrieval targets: the document corpus (what does the knowledge base say?) and the memory store (what do we know about this user's current state?). **Memory isn't a separate system bolted on the side — it's another collection in your Collector, another signal in your Ranker, and another source of context in your Server.**

Memory in the C→R→S framework:

- **Collector:** stores user-scoped observations with timestamps and metadata (not just document embeddings)
- **Ranker:** uses recency, preference history, and temporal validity as ranking signals
- **Server:** assembles retrieved documents *and* user memories into the context

Agents as Routers

With multiple representations in the Collector, the Server must decide which one to search for each query. This is the **routing problem.**

- For small route sets (5–10 index types): LLM classifies via tool call — the agentic search pattern from Unit 6.
- For larger route sets: a fine-tuned classifier is more appropriate.

Glean — the Server with permission-aware routing

Glean provides enterprise search across 100+ SaaS tools (Slack, Google Drive, Jira, Confluence, Salesforce, etc.):

- **Routing across 100+ sources:** each SaaS connector has its own index with different document schemas, update frequencies, and access patterns. The Server decides which sources to query for each request.
- **Personal graph:** models individual employee work patterns — which tools they use, which documents they access, who they collaborate with. Non-query signal personalization implemented as a graph.
- **Enterprise graph:** models company entities and relationships — org structure, project ownership, team membership. Used to augment relevance with organizational context.
- **Permission enforcement at retrieval time:** enforces permissions *during* retrieval, not after — otherwise the system leaks information through relevance signals even without showing content.

RAG vs. Tool Use: Where's the Line?

	RAG	Tool use
What the model receives	Retrieved documents in context	Tool result as a message
Who decides what to fetch	The retrieval pipeline (Collector + Ranker)	The model (via function calling)
When it happens	Before generation (context pre-assembled)	During generation (model decides mid-response)
Best for	"Answer this question using these documents"	"Look up this specific fact" or "take this action"
Eval focus	Retrieval quality (recall, coverage, diversity)	Tool selection accuracy, argument correctness

In practice, the two **compose**. An agentic RAG system uses tool use to trigger retrieval: the model calls a `search` tool (Unit 5's function calling), the tool runs the C→R→S pipeline, and results come back as context for the next generation step. Glean's 100+ source routing is exactly this.

P^3 implication

RAG and tool use have **different proof requirements.** For RAG: retrieval quality + faithfulness. For tool use: tool selection accuracy + argument correctness. A system combining both needs eval for both — don't let one hide failures in the other.

Diversity and Coverage: The Recsys Lesson

In recsys, recommending 10 variations of the same item is a known failure mode. Diversity constraints (*BPRS* Ch. 15) ensure the final set covers multiple aspects of the user's interests.

In RAG, the equivalent failure: retrieving 10 documents that all say the same thing. The LLM doesn't get the evidence breadth it needs.

FreshStack formalizes this with three evaluation dimensions:

Dimension	Metric	What it measures
Relevance	Recall@50	Are the docs on-topic?
Grounding (Coverage)	Coverage@20	What % of unique atomic facts ("nuggets") are supported?
Diversity	alpha-nDCG@10	Non-redundancy — penalize retrieving docs that repeat the same fact

Coverage is especially important: even if you retrieve 10 relevant documents, if they all support the same nugget and miss others, the LLM generates an incomplete answer.

Key FreshStack finding: **no single model performs best across all topics.** The gap between current performance and the theoretical maximum is large. This reinforces why you need domain-specific eval rather than trusting a leaderboard.

Graph Structures in Retrieval: A Primer

A graph is a collection of **nodes** (entities) connected by **edges** (relationships). Nodes can be documents, users, concepts, products. Edges encode a relationship: "cites," "was-accessed-together," "shares-an-author," "co-purchased."

Homogeneous graph

Only one type of node, one type of edge. Often more naturally represented as a similarity matrix or an ANN index.

Note: **HNSW is itself a navigable graph** over embedding vectors. You already have a graph.

Heterogeneous graph

Different types of nodes and edges — documents, authors, topics, organizations — with authorship, topic membership, and citation edges. Querying across these simultaneously is where graphs outshine flat vector search.

PinSage (Ying et al., 2018 — Pinterest)

Pins are nodes; edges encode co-saves (if many users saved both pin A and pin B, they're connected). Lets the system recommend pin A not because it *looks like* what you searched for, but because users who engaged with your kind of content also engaged with it — the relational signal that no single-item embedding captures.

The same logic applies to documents: a document might not be semantically close to your query, but it co-occurs with relevant documents in citations, link structure, or access patterns. Graph structure surfaces these connections.

Link Retrieval: Neighbor and Bridging Patterns

Neighbor retrieval (the cluster hypothesis): documents similar to a relevant document are also likely relevant.

Neighbor retrieval

```
Query → [Retrieve top-k] → [Rerank to top-3]
      → [Retrieve neighbors of top-3] → final candidate set
```

Graph traversal using your existing vector index. Walk one hop through the similarity graph. Useful when the corpus has dense clusters and the initial query might only land in the neighborhood, not on the exact target.

Bridging retrieval: use one representation to find a document, then use a *different relationship* to find what the LLM actually needs.

Example

A user asks a vague question about quarterly performance. Semantic search finds an analyst memo closely related to the query in embedding space. That memo *cites* a specific financial report. Follow that citation link to retrieve the actual report. **The memo was the bridge; the report is what the LLM needs.**

In recsys terms: the memo is the "gateway item" — similar to the user's expressed intent, but the thing actually useful for completing the task is one relational hop away.

Multi-Hop Retrieval

Multi-hop retrieval is the natural extension of bridging retrieval: chain multiple hops to build the evidence needed for a complex question.

Example: vendor renegotiation

"Given our current vendor contracts and last quarter's spend data, which vendors should we renegotiate with this cycle?"

1. Retrieve vendor contracts — current terms and expiry dates
2. Retrieve spend data — identify highest-cost vendors
3. Join: vendors with unfavorable terms *and* high spend
4. Retrieve recent performance reviews for those vendors

No single retrieval pass can get all of this. Each step's result informs the next query. This chains 3–4 dependent retrieval steps.

Key engineering concern: context compounds per hop

After three hops you have three sets of documents in context — context poisoning risk rises with each one. **Subagents** (Unit 5) are the structural answer: each hop runs in its own clean context, and only the distilled finding passes to the next hop or back to the orchestrator.

The agentic search pattern from Unit 5 handles this naturally: the LLM issues a search, reads results, decides

When to Use a Graph Database?

Rarely.

Both neighbor retrieval and bridging retrieval can be implemented with a vector store plus structured metadata (citation links, access-pattern co-occurrence, document relationships stored as JSON or in Postgres).

Purpose-built graph databases (Neo4j, etc.) are worth adding **only when** you have complex multi-hop traversal at scale where SQL or in-memory graphs become too slow.

For most RAG systems:

- The cluster hypothesis can be exploited with your existing vector index
- Explicit link structures can live in a standard relational store alongside your embeddings
- **When you only have 300 documents, just put them all in context**

P^3 lens

The full RAG system's **Promise** is "grounded, accurate answers over private data." The **Proof** requires measuring at every stage: Collector recall, Ranker precision/NDCG, and Server faithfulness/coverage. A single end-to-end metric hides which component is failing — the "metric laundering" pitfall from Unit 3. In **Production**, these per-stage metrics become monitoring signals: if Collector recall drops (new doc types not being indexed), you catch it before end-to-end quality degrades.

Outline

1. Recap: Unit 6
2. How This Connects
3. What Is a Recommendation System?
4. A Brief History
5. The Collector: Generating Representations
6. The Ranker: Matching Intent to Documents
7. The Server: From Ranked Results to Grounded Answers
8. Putting It All Together

"Why Do People Say RAG Is Dead?"

"RAG is dead" means the **2023-era approach** of embedding every document as a single vector and doing cosine similarity — popularized by DevRels and content creators — is insufficient.

That approach has real problems: information loss from pooling, training distribution mismatch, no diversity or coverage.

Everything in this lecture is a response to one or more of those problems.

"RAG is dead" means naive single-vector RAG is insufficient. It doesn't mean retrieval is dead.

Retrieval Evaluation: Your Own Journey

The leaderboards (BEIR, MTEB) are useful for orientation, but they're contaminated — models are trained on these benchmarks now. **The only eval that matters is the one you build on your data.**

This is Unit 3's lesson applied to retrieval. The pattern:

1. **Build a golden dataset.** 50–100 real or realistic queries from your actual use case. Cover typical queries, long-tail queries, and the hard cases where you expect failure. This is the most important investment; everything else depends on it.
2. **Define your failure modes.** Before you run anything, write down what "bad retrieval" looks like. Missing a critical document? Returning 10 copies of the same fact? Retrieving the right document but the wrong chunk? Each failure mode suggests a different metric.
3. **Establish baselines.** BM25 alone. Dense alone. Hybrid. Run all three on your golden dataset before adding any complexity. You'll be surprised how often BM25 is competitive — and you need to know the baseline before you can claim a win.
4. **Iterate with evidence.** Add a new chunking strategy? Measure it. Switch embedding models? Measure it. Add a late-interaction reranker? Measure it. Systematic measurement is what lets you trust your conclusions.
5. **Use multiple metrics.** Precision@K, recall@K, and an LLM judge. A chunk can be "relevant" by nDCG but lack enough context to actually answer the question.
6. **Feed failures back.** Every time the system returns a bad result, add the query to your eval set. The golden dataset should grow with your system, not sit frozen while production diverges from it.

Key Takeaways (1–6)

1. **RAG is a recommendation system.** Collector → Ranker → Server. The recsys playbook applies directly: user modeling, item modeling, multi-stage ranking, diversity, cold-start. The three responsibilities organize every advanced technique.
2. **You can only find what you've made findable.** Representation engineering is the discipline of deliberately shaping how items and queries are encoded. This is the Collector's core job — and it's more important than your choice of vector index.
3. **Chunking is feature engineering.** Every chunking decision is a decision about what knowledge can be inferred from any atomic unit. Expert chunking: prepend situation context, follow document structure, use inconsistent chunk sizes based on semantic coherence. The "right" chunk size is a hyperparameter — measure it on your corpus.
4. **Pre-filter before you search, not after.** Post-filtering can yield zero results even when the answer exists. Pre-filtering narrows the search space so the ANN search focuses its budget on the right region. Every enriched representation is a potential new filtering dimension.
5. **Pooling is the fundamental flaw of single-vector search.** Late interaction (ColBERT / MaxSim) keeps token-level detail and beats dense models with the same backbone — often by 60%+ on reasoning tasks. A 150M-parameter late interaction model outperforms all 7B-parameter dense models on reasoning tasks.
6. **Rank for the LLM, not for a human.** Lost-in-the-middle means ordinal rank isn't what you want. The best documents should be at the beginning or end of context. Be selective about K. Annotate highly relevant documents explicitly. Diversity matters more than stacking the same top-ranked documents.

Key Takeaways (7–11)

7. **Context rot and context poisoning are real.** More context makes models worse, especially under distractors. A single misleading token can send the model down the wrong path. Retrieve less but better. The Chroma research confirms: no single model resists context rot across all tasks.
8. **Memory is another collection in your RAG system.** A complete RAG system retrieves from both the document corpus and the user's memory store. Memory isn't a separate paradigm — it's stateful, user-scoped retrieval with temporal validity. The database migration example shows why semantic similarity fails for temporally invalidated facts.
9. **Graphs are already in your retrieval stack.** HNSW is a graph. Neighbor retrieval (cluster hypothesis) and bridging retrieval (gateway documents) give you graph-like patterns with your existing vector index. You rarely need a separate graph database — and when you only have 300 documents, just put them all in context.
10. **Interleaving weights are hyperparameters — tune them on your eval set.** With two indices, you have one weight to tune. With five, you have a five-dimensional space. The more representations you create, the more dependent you become on systematic measurement.
11. **Build your own eval and iterate.** Golden dataset → baselines → systematic experiments → LLM judge + IR metrics → feed failures back. The leaderboards don't know your corpus. Ship hybrid BM25 + dense, measure, improve.

Further Reading

Course texts:

- Bischof & Yee, *Building Production Recommendation Systems in Python and JAX* (O'Reilly, 2024) — Ch. 1–4 (C→R→S), Ch. 2–3 (user-item matrix, CF), Ch. 7 (cold-start), Ch. 12 (multi-stage), Ch. 14–15 (diversity)
- Husain, Clavié, Thakur, Weller, Chaffin, Bischof et al., *Beyond Naive RAG: Practical Advanced Methods* (2025) — `resources/beyond-naive-rag.pdf`

Key papers:

- Karpukhin et al., DPR (2020) `arxiv.org/abs/2004.04906`; Lewis et al., RAG (2020) `arxiv.org/abs/2005.11401`
- Khattab & Zaharia, ColBERT (2020) `arxiv.org/abs/2004.12832`; Izacard & Grave, FiD (2021) `arxiv.org/abs/2007.01282`
- Weller et al., Promptriever (2024) `arxiv.org/abs/2409.11136`; Weller et al., Rank1 (2025) `arxiv.org/abs/2502.18418`
- Ying et al., PinSage (2018) `arxiv.org/abs/1806.01973`; Gao et al., HyDE (2022) `arxiv.org/abs/2212.10496`
- Malkov & Yashunin, HNSW (2016) `arxiv.org/abs/1603.09320`; Thakur et al., FreshStack (2025)
- Sarthi et al., *RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval* (ICLR 2024) — `arxiv.org/abs/2401.18059`
- Faysse et al., *ColPali: Efficient Document Retrieval with Vision Language Models* (2024) — `arxiv.org/abs/2407.01449`
- Kusupati et al., *Matryoshka Representation Learning* (NeurIPS 2022) — `arxiv.org/abs/2205.13147`

Production case studies:

- Bischof et al., *Semantic Dot Art* — `semantic.art` · LanceDB technical blog
- Notion, *Two Years of Vector Search: 10× Scale, 1/10th Cost* (Feb 2026) — `notion.com/blog/two-years-of-vector-search-at-notion`
- Instacart, *Building the Intent Engine* — `instacart.com/.../building-the-intent-engine`
- Glean, *Unified Index for Enterprise AI* — `glean.com/product/system-of-context`
- Hong et al., *Context Rot* (July 2025) — `research.trychroma.com/context-rot`

Thank you!